

Outlines

- Introduction
- Function Template
- Function Template with Multiple Template type
- Class Template

Introduction

Templates are powerful features of C++ that allows us to write generic programs.

It allows us to define the generic classes and generic functions and thus provides support for generic programming.

Generic programming is a technique where generic types are used as parameters in algorithms so that they can work for a variety of data types.

Generic Programming is typically used when we want to write code that is not specific to a particular data type and can work with a variety of data types.

There are two ways we can implement templates:

- Function Templates
- Class Templates

Why Templates are Used in Programming

Code Reusability: Allows the same code to work with different data types.

Type Safety: Ensures type errors are caught at compile time.

Reduced Code Duplication: Eliminates the need to write multiple versions of functions or classes for different data types.

Generic Programming: Facilitates the creation of algorithms and data structures that work with any data type.

Function Template

We can create a single function to work with different data types by using a function template.

For example, if we have an `add()` function, we can create versions of the `add` function for adding the `int`, `float` or `double` type values.

A function template starts with the keyword **template** followed by template parameter(s) inside `<>` which is followed by the function definition.

Basic idea: Create a function without having to specify the exact type(s) of some or all of the variables. Instead, we define the function using placeholder types, called template type parameters.

Function Template Declaration

```
template <typename T>  
T functionName (T parameter1, T parameter2, ...) {  
    // code  
}
```

OR

```
template <class T>  
T functionName (T parameter1, T parameter2, ...) {  
    // code  
}
```

In the above code, **T** is a template argument that accepts different data types (**int**, **float**, etc.), and **typename** or **class** is a keyword.

Calling a Function Template

```
functionName<dataType>(parameter1, parameter2, ...);
```

For example, let us consider we need to add two numbers.

```
int Add(int num1 , int num2) {  
    return num1 + num2;  
}
```

```
double Add (double num1, double num2) {  
    return num1 + num2;  
}
```

Rewriting the same function body over and over for different types is time-consuming and wastes space in the listing.

Also, if you find you've made an error in one such function, you'll need to remember to correct it in each function body.

Function templates let us write such a function just once, and have it work for many different data types.

Function Template Example

```
#include <iostream>
using namespace std;

template <typename T>
T add(T num1, T num2) {
    return (num1 + num2);
}
```

```
int main() {
    int result1;
    double result2;

    // calling with int parameters
    result1 = add<int>(2, 3);
    cout << result1 << endl;

    // calling with double parameters
    result2 = add<double>(2.2, 3.3);
    cout << result2 << endl;
    return 0;
}
```

```
#include<iostream>
```

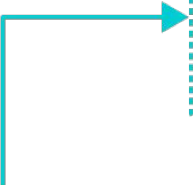
```
template<typename T>  
T add(T num1, T num2) {  
    return (num1 + num2);  
}
```

```
int main() {  
    ... ..
```

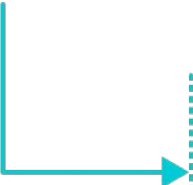
```
    result1 = add<int>(2,3);  
    ... ..
```

```
    result2 = add<double>(2.2,3.3);  
    ... ..
```

```
}
```



```
int add(int num1, int num2) {  
    return (num1 + num2);  
}
```



```
double add(double num1, double num2) {  
    return (num1 + num2);  
}
```

Function templates with multiple template types

Definition and Purpose

- Function templates can accept multiple template arguments to operate with multiple data types.
- They allow creating functions that can work with different combinations of data types.

Example Program

```
#include <iostream>
using namespace std;

// function template
template <typename T, typename U>
T add(T a, U b){
    return a + b;
}

int main() {
    cout << "Result:" << add(5, 5.5) << endl;
    // Output: 10
}
```

Here the return type will be set to **int** so, the decimal part will be ignored. To solve it, **auto** type can be used.

```
#include <iostream>
using namespace std;

// function template
template <typename T, typename U>
auto add(T a, U b){
    return a + b;
}

int main() {
    cout << "Result:" << add(5, 5.5) << endl; //
    Output: 10.5
}
```

Class Template

Similar to function templates, we can use class templates to create a single `class` to work with different `data types`.

Class templates come in handy as they can make our code shorter and more manageable.

Class Template Declaration

A class template starts with the keyword **template** followed by template parameter(s) inside **<>** which is followed by the class declaration.

```
template <class T>
class className {
    private:
        T var;
        // code
    public:
        T functionName(T arg);
        // code
};
```

In the above declaration, **T** is the template argument which is a placeholder for the data type used, and **class** is a keyword.

Creating a Class Template Object

```
className<dataType> classObject;
```

For example:

```
className<int> classObject;
```

```
className<float> classObject;
```

```
className<string> classObject;
```

Class Template Example

```
#include <iostream>
using namespace std;
// Class template
template <class T>
class Number {
    private:
        // Variable of type T
        T num;

    public:
        Number(T n){ // constructor
            num = n;
        }
        T getNum() {
            return num;
        }
};
```

```
int main() {

    // create object with int type
    Number<int> numberInt(7);

    // create object with double type
    Number<double> numberDouble(7.7);

    cout << "int Number = " << numberInt.getNum() <<
endl;
    cout << "double Number = " <<
numberDouble.getNum() << endl;

    return 0;
}
```

Class Template with Multiple Parameters

we can use multiple template parameters and even use default arguments for those parameters.

```
template <class T, class U, class V = int>
class ClassName {
    private:
        T member1;
        U member2;
        V member3;
        ... ..
    public:
        // code
};
```

Example Program - class template with multiple parameters

```
#include <iostream>
using namespace std;

// Class template with multiple and default
parameters
template <class T, class U, class V = char>
class ClassTemplate {
private:
    T var1;
    U var2;
    V var3;

public:
    ClassTemplate(T v1, U v2, V v3) : var1(v1),
var2(v2), var3(v3) {} // constructor

    void printVar() {
        cout << "var1 = " << var1 << endl;
        cout << "var2 = " << var2 << endl;
        cout << "var3 = " << var3 << endl;
    }
};
```

```
int main() {
    // create object with int, double and char
types
    ClassTemplate<int, double> obj1(7, 7.7, 'c');
    cout << "obj1 values: " << endl;
    obj1.printVar();

    // create object with int, double and bool
types
    ClassTemplate<double, char, bool> obj2(8.8,
'a', false);
    cout << "\nobj2 values: " << endl;
    obj2.printVar();

    return 0;
}
```

Q. (April/May 2023)

Write a class template which sorts and returns the average of values stored in an array.

```

#include <iostream>
#include <algorithm> // For sort
using namespace std;

template <typename T>
class ArrayProcessor {
private:
    T* data;
    int size;
public:
    ArrayProcessor(T arr[], int s) {
        data = arr;
        size = s;
    }
    void sort_data() {
        sort(data, data + size);
    }

```

```

    T average() {
        T sum = 0;
        for (int i = 0; i < size; ++i) {
            sum += data[i];
        }
        return sum / size;
    }
    void print() {
        for (int i = 0; i < size; ++i) {
            cout << data[i] << " ";
        }
        cout << endl;
    }
};

```

Main Program:

```
int main() {  
    int intArr[] = {5, 3, 8, 6, 2};  
    int intSize = 5;  
    ArrayProcessor<int> intProcessor(intArr, intSize);  
    intProcessor.sort_data();  
    intProcessor.print(); // Output: 2 3 5 6 8  
    cout << "Average: " << intProcessor.average() << endl; // Output: Average: 4  
  
    double doubleArr[] = {3.5, 2.1, 4.8, 5.6};  
    int doubleSize = 4;  
    ArrayProcessor<double> doubleProcessor(doubleArr, doubleSize);  
    doubleProcessor.sort_data();  
    doubleProcessor.print(); // Output: 2.1 3.5 4.8 5.6  
    cout << "Average: " << doubleProcessor.average() << endl; // Output: Average: 4  
  
    return 0;  
}
```



Any Queries?